

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Aswanth N N	Reg. No.:	192424123
Section / Batch:	2024	Date:	12-08-2026

Code of Conduct

I Aswanth N N (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Ans:

The question deals with morphological decomposition of treatment, treatable, retreatment, treated, and untreated and asks for correction of incorrect analyses.

1. Words producing incorrect morphological analysis

The likely problematic words are:

- treatment
- treatable
- retreatment
- treated
- untreated

A simple stemmer may incorrectly remove parts such as ment, able, re, un, or ed without understanding their morphological function.

2. Correct morphological decomposition

Word	Prefix	Root/Base	Suffix	Type
treatment	—	treat	-ment	Derivational
treatable	—	treat	-able	Derivational
retreatment	re-	treat	-ment	Both derivational
treated	—	treat	-ed	Inflectional
untreated	un-	treat	-ed	un- derivational, -ed inflectional

3. Morphological interpretation

Treatment

treat + ment → treatment

- treat = root/base
- -ment = derivational suffix
- Changes a verb into a noun.
- Example: *treat* → *treatment*

Treatable

treat + able → treatable

- -able is derivational.
- It produces an adjective meaning "capable of being treated."

Retreatment

re + treat + ment → retreatment

- re- = derivational prefix meaning "again"
- -ment = derivational suffix
- Contains two derivational layers.

Treated

treat + ed → treated

- -ed is an inflectional suffix.
- It indicates past tense/past participle.
- It does not create a new lexical category.

Untreated

un + treat + ed → untreated

- un- = derivational prefix indicating negation
- -ed = inflectional suffix
- Therefore, it has both derivational and inflectional morphology.

4. Root cause of the error

The main problem is that the system performs surface-level suffix/prefix removal rather than understanding morphological structure.

For example, treating:

treated → treat

is reasonable for normalization, but treating every occurrence of -ed or -ment as removable without considering context can destroy useful biomedical information.

Similarly:

retreatment → treatment

would lose the important meaning "again", while:

untreated → treated

would incorrectly remove the negation.

5. Corrected strategy

A better morphological analyzer should:

1. Identify prefixes.
2. Identify the lexical root.
3. Identify suffixes.
4. Classify each affix as inflectional or derivational.
5. Preserve important semantic prefixes such as un- and re-.
6. Apply rules according to word structure rather than simple string deletion.

6. Original vs corrected analysis

Word	Incorrect approach	Correct approach
treatment	treat	treat + ment
treatable	treat	treat + able
retreatment	treatment	re + treat + ment
treated	treat + ?	treat + ed
untreated	treat	un + treat + ed

7. Effect on biomedical information retrieval

Correct morphological analysis improves retrieval because medically meaningful distinctions are preserved.

For example:

- untreated cancer should not be treated exactly like treated cancer.
- retreatment indicates treatment again.
- treatment and treatable are morphologically related but have different meanings.

Therefore, the corrected system can improve precision and recall by grouping related terms while preserving important semantic distinctions.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Ans:

The question gives the words replayed, unhappier, disconnected, players, restarting, unreadable and asks for FSA/FST error analysis.

1. Correct morphological analysis

Word	Correct decomposition
replayed	re + play + ed
unhappier	un + happy + er
disconnected	dis + connect + ed
players	play + er + s
restarting	re + start + ing
unreadable	un + read + able

2. Common parser error

The parser can recognize a single affix, but fails when multiple affixes occur in the same word.

For example, it may correctly recognize:

play + ed

but fail with:

re + play + ed

because the FSA has no transition that allows another affix after the first one.

3. Incorrect transition

A simplified incorrect FSA may behave like:

START

|

|-- Prefix --> ROOT

|

|-- Suffix --> END

This allows only:

prefix + root

or

root + suffix

but not:

prefix + root + suffix

or:

prefix + root + suffix + suffix

4. Corrected transition structure

The parser should allow multiple affixes:

START

|

v

PREFIX*

|

v

ROOT

|

v

SUFFIX*

|

v

END

For example:

START → re → play → ed → END

For players:

START → play → er → s → END

For unreadable:

START → un → read → able → END

5. Corrected parser output

Word	Prefix	Root	Suffixes
replayed	re-	play	-ed
unhappier	un-	happy	-er
disconnected	dis-	connect	-ed
players	—	play	-er, -s
restarting	re-	start	-ing
unreadable	un-	read	-able

6. Accuracy calculation

Parsing accuracy is:

$$\text{Accuracy} = \frac{\text{Correctly\ Parseds\ Words}}{\text{Total\ Words}} \times 100 \%$$

For example, if 80 out of 100 words are correctly parsed before correction:

$$\text{Accuracy}_{\text{before}} = \frac{80}{100} \times 100 = 80\%$$

If 94 are correctly parsed after correction:

$$\text{Accuracy}_{\text{after}} = \frac{94}{100} \times 100 = 94\%$$

Therefore:

$$\text{Improvement} = 94 - 80 = 14 \text{ percentage points}$$

7. Limitations

- English morphology has many exceptions.
 - Some words have ambiguous analyses.
 - The number of FST states/transitions can increase as rules become more complex.
 - Irregular forms are difficult to handle using only rules.
 - Social-media text contains spelling errors, abbreviations and slang.
-

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

Ans:

The given program is:

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The assignment specifically asks to identify preprocessing errors and correct tokenization and stemming.

1. Programming/preprocessing errors

The main error is:

```
data["Text"].apply(ps.stem)
```

PorterStemmer.stem() expects one word/token at a time, but data["Text"] contains complete sentences or documents.

For example:

"The players are playing football"

is passed as one string instead of individual tokens.

2. Why the output is incorrect

The correct process should be:

Document



Sentence/Text



Tokenization



Individual words



Stemming



Stemmed tokens

Instead, the original program does:

Complete sentence



PorterStemmer



Incorrect processing

3. Corrected program

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import pandas as pd
```

```
ps = PorterStemmer()
```

```
data = pd.read_csv("news.csv")
```

```
def preprocess(text):
    tokens = word_tokenize(str(text))
    stems = [ps.stem(token) for token in tokens]
    return stems
```

```
data["Tokens"] = data["Text"].apply(
    lambda x: word_tokenize(str(x))
)

data["Stemmed"] = data["Tokens"].apply(
    lambda tokens: [ps.stem(token) for token in tokens]
)

print(data[["Text", "Tokens", "Stemmed"]].head())
```

If punctuation should be excluded:

```
def preprocess(text):
    tokens = word_tokenize(str(text))
    tokens = [t.lower() for t in tokens if t.isalpha()]
    stems = [ps.stem(t) for t in tokens]
    return stems
```

4. Example output

Original:

“The players are playing games.”

Tokens:

['The', 'players', 'are', 'playing', 'games']

Stemmed tokens:

['the', 'player', 'are', 'play', 'game']

5. Original vs corrected

Original text	Tokens	Stemmed tokens
The players are playing games	the, players, are, playing, games	the, player, are, play, game
Students studied NLP	students, studied, NLP	student, studi, nlp
Connected systems are connecting	connected, systems, are, connecting	connect, system, are, connect

6. Twenty problematic stemming examples

Word	Porter stem	Type
------	-------------	------

studies	studi	Inflectional
studied	studi	Inflectional
studying	studi	Inflectional
study	studi	Base
connected	connect	Inflectional
connecting	connect	Inflectional
connections	connect	Inflectional
connection	connect	Derivational
organization	organ	Derivational
organizational	organ	Derivational
organize	organ	Derivational
organized	organ	Inflectional
organizer	organ	Derivational
players	player	Inflectional
player	player	Derivational
playing	play	Inflectional
played	play	Inflectional
happiness	happi	Derivational
happily	happili	Derivational
readable	readabl	Derivational

7. Improvement

The corrected program separates:

Tokenization → Normalization → Stemming

This prevents complete sentences from being incorrectly sent to the stemmer and provides cleaner morphological preprocessing for NLP tasks.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

```
words = [
```

```

    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

```

```

for w in words:
    print(w, "->", parser(w))

```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Ans :

The original program is:

```

def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

```

The question asks for corrections covering regular, -es, -ies, and irregular plurals.

1. Logical errors

Error 1: Incorrect slicing

word[:-2]

is always removing two characters.

For:

cars

it produces:

ca

But the correct singular is:

car

It should remove only one character for regular -s plurals.

Error 2: No -es rule

For:

boxes

the singular is:

box

The program does not specifically handle -es.

Error 3: No -ies rule

For:

cities

the singular is:

city

The parser needs:

ies → y

Error 4: No irregular plural handling

For:

children

there is no s, so the original program incorrectly returns:

children → Singular

2. Corrected parser

def parser(word):

```
    irregular = {  
            "children": "child",  
            "men": "man",  
            "women": "woman",  
            "mice": "mouse",  
            "feet": "foot",  
            "teeth": "tooth"  
    }
```

```
if word in irregular:
    return irregular[word], "Plural Noun"
```

```
if word.endswith("ies"):
    return word[:-3] + "y", "Plural Noun"
```

```
if word.endswith(("ses", "xes", "zes", "ches", "shes")):
    return word[:-2], "Plural Noun"
```

```
if word.endswith("s") and not word.endswith("ss"):
    return word[:-1], "Plural Noun"
```

```
return word, "Singular"
```

3. Correct output

cars → car → Plural Noun
boxes → box → Plural Noun
cities → city → Plural Noun
children → child → Plural Noun
books → book → Plural Noun

Comparison

Word	Original output	Correct output
cars	ca, Plural	car, Plural
boxes	box, Plural	box, Plural
cities	cit, Plural	city, Plural

children	children, Singular	child, Plural
books	book, Plural	book, Plural

4. Why the original fails

English plural formation is not governed by one rule.

There are several patterns:

book → books

box → boxes

city → cities

child → children

Therefore, a single:

endswith("s")

condition is insufficient.

5. Limitations

A rule-based FSA/FST:

- Cannot easily handle every irregular noun.
- Requires many manually created rules.
- Can produce ambiguity.
- May fail on unknown words.
- Does not understand semantic context.
- Becomes more complicated as morphological rules increase.

6. Improvement

The corrected parser explicitly recognizes different plural formation rules, producing better morphological analyses and reducing false singular/plural classifications.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
```

```

        "studies studied studying study",
        "organize organized organizer organization"
    ]

    vectorizer = CountVectorizer()

    X = vectorizer.fit_transform(documents)

    features = [
        stemmer.stem(word)
        for word in vectorizer.get_feature_names_out()
    ]

```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Ans:

The question provides a program that first performs CountVectorizer feature extraction and only then applies Porter stemming.

1. Preprocessing error

The original code:

```

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]

```

extracts the vocabulary before normalization.

Therefore, the vectorizer creates separate features for:

connected

connection

connecting

connectivity

and only afterward are these features stemmed.

The feature matrix X still contains the original vocabulary.

2. Why this is inappropriate

The correct order is:

Raw documents



Tokenization



Normalization/Stemming



Feature extraction



Feature matrix



Machine-learning model

The original order is:

Raw documents



Feature extraction



Stemming vocabulary

This means the stemmed feature list does not actually modify X.

3. Corrected program

```
from nltk.stem import PorterStemmer
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
stemmer = PorterStemmer()
```

```
documents = [  
    "connected connection connecting connectivity",  
    "studies studied studying study",  
    "organize organized organizer organization"  
]
```

```
def stem_text(text):  
    words = text.lower().split()  
    return " ".join(stemmer.stem(word) for word in words)
```

```
normalized_documents = [  
    stem_text(doc)  
    for doc in documents  
]
```

```
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(normalized_documents)  
print("Normalized documents:")  
print(normalized_documents)  
print("\nVocabulary:")  
print(vectorizer.get_feature_names_out())  
print("\nFeature matrix:")  
print(X.toarray())
```

4. Example normalization

Document 1

Original:

connected connection connecting connectivity

After stemming:

connect connect connect connect

Document 2

Original:

studies studied studying study

After stemming:

studi studi studi studi

Document 3

Original:

organize organized organizer organization

After stemming:

organ organ organ organ

5. Before vs after

Aspect	Before correction	After correction
Normalization	After feature extraction	Before feature extraction
Vocabulary	Contains variants	Contains normalized stems
connect* words	Multiple features	One feature
studi* words	Multiple features	One feature
organ* words	Multiple features	One feature
Feature matrix	Original words	Normalized words

Vocabulary example

Before:

connected
connecting
connection
connectivity
studied
studies
studying
study

After:

connect
studi
organ

Thus, the vocabulary becomes smaller and related morphological forms can share the same feature.

6. Ten normalization examples

Original	Stemmed
----------	---------

connected	connect
connection	connect
connecting	connect
connectivity	connect
studies	studi
studied	studi
studying	studi
study	studi
organized	organ
organization	organ

7. Effect on classification

The corrected pipeline can:

- Reduce vocabulary size.
- Reduce dimensionality.
- Combine related word forms.
- Reduce redundant features.
- Improve computational efficiency.
- Potentially improve classification generalization.

However, stemming can sometimes over-stem words and remove distinctions that are useful for classification.

Important experimental metrics

The assignment asks for:

- Vocabulary size before correction
- Vocabulary size after correction
- Classification accuracy before correction

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors

Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____